CS50

Week 3

Algorithms

Outlines :

- ⟶ Sorting
- ⟶ Selection Sort
- ⟶ Bubble Sort
- ⟶ Insertion Sort

Sorting :

Definition

Sorting is the process of arranging data in a specific order, typically either ascending or descending. This can apply to numbers, strings

Advantages

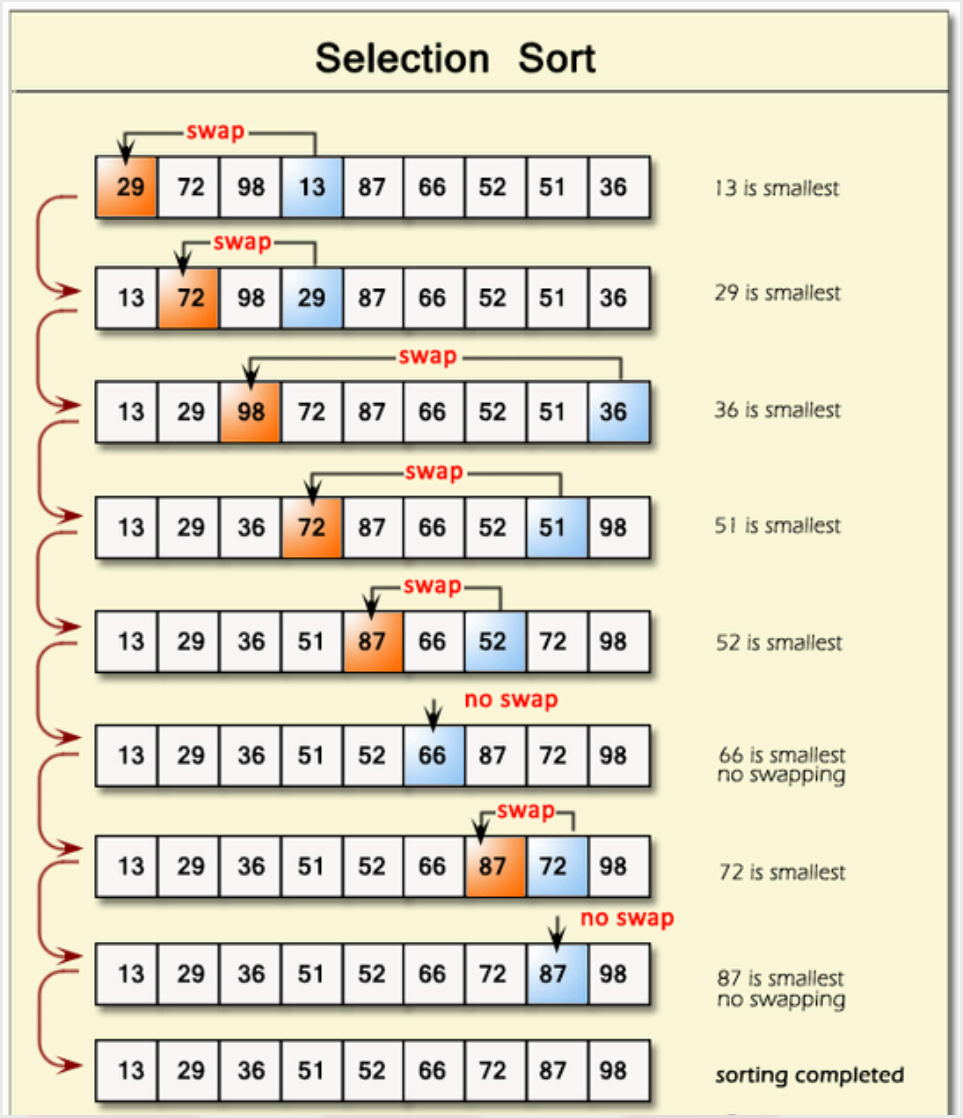
- Improved Efficiency
- Data Organization
- Data Analysis
- Duplicate Identification
- Visualization

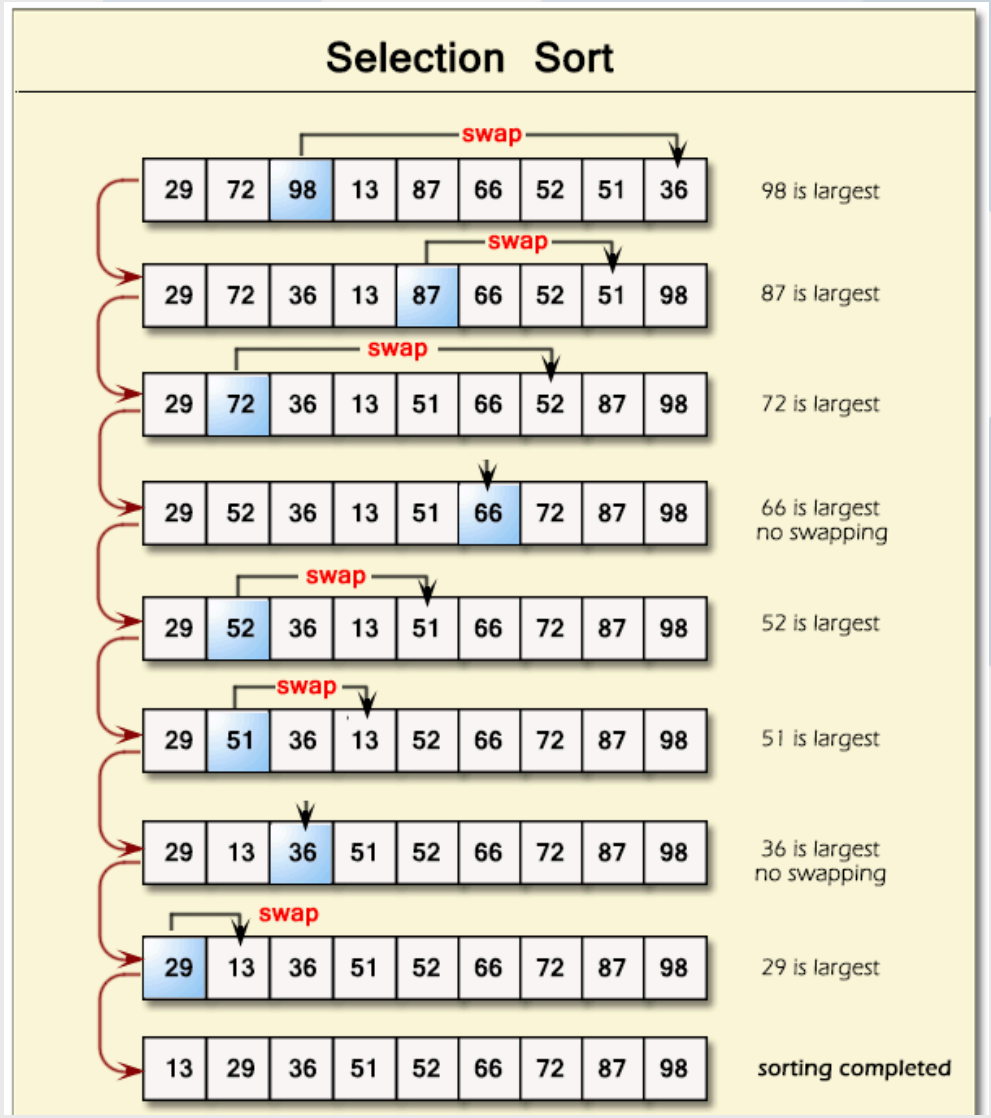
Selection

Bubble

Insertion

Selection Sort :





Selection Sort :

- The algorithm for selection sort in pseudocode is:

```
For i from 0 to n-1
    Find smallest number between numbers[i] and numbers[n-1]
    Swap smallest number with numbers[i]
```

(Pseudocode)

Selection Sort	$O(n^2)$	$\Omega(n^2)$
	Worst Case	**Best Case**

Selection Sort Code : (Example#1)

```cpp
#include <iostream>
using namespace std;

void SelectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        swap(arr[i], arr[minIndex]);
    }
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
}

int main() {
    int arr[] = {2, 1, 3, 1, 2};
    int n = 5;
    SelectionSort(arr, n);
    return 0;
}
```

Selection Sort : (Example#2)

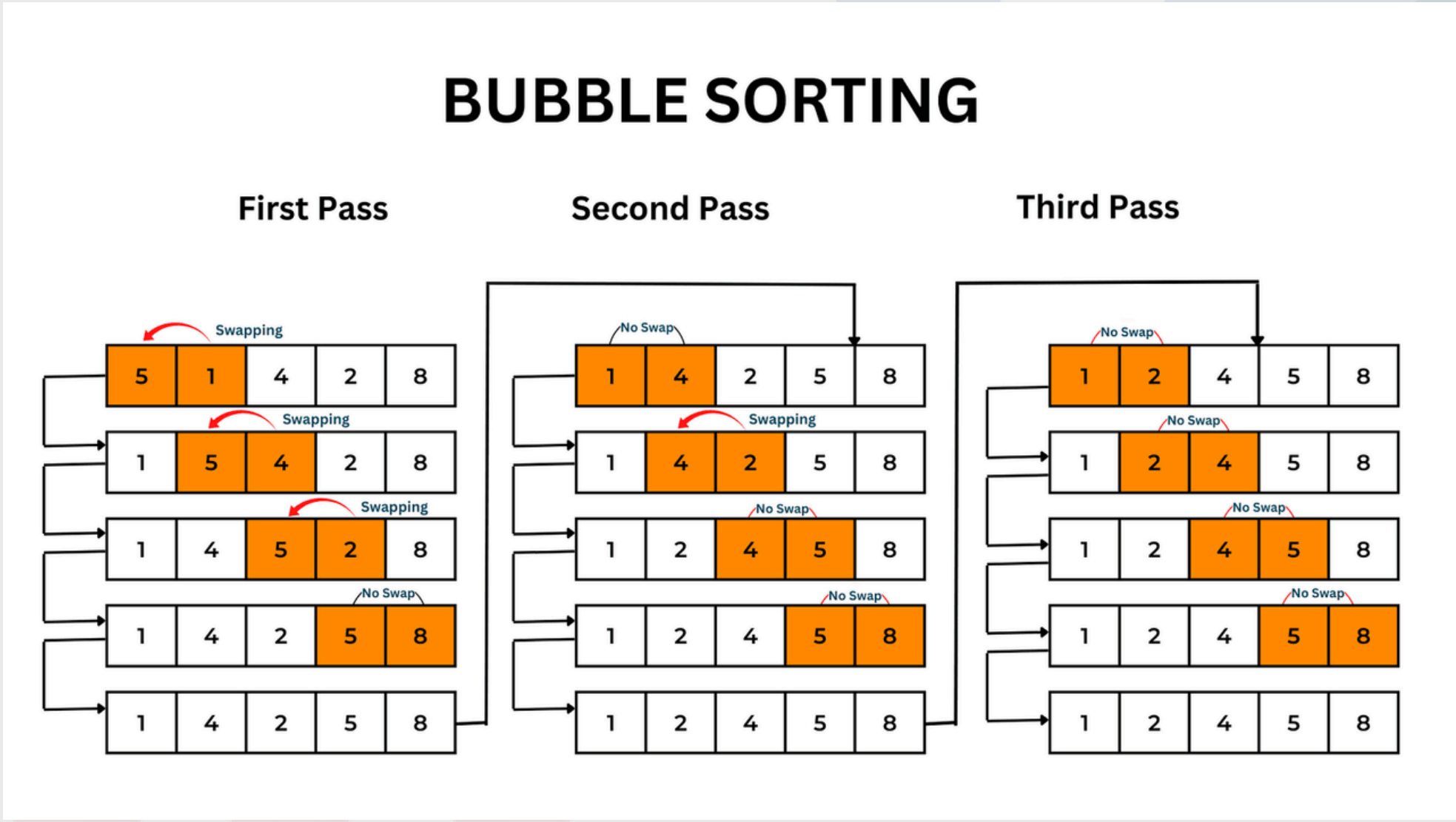
Find Largest and Smallest Element in an array

```cpp
#include <iostream>
using namespace std;

void FindLargetAndSmallest(int arr[], int n) {
    int largest = arr[0];
    int smallest = arr[0];
    for (int i = 1; i < n; i++) {
        if (arr[i] > largest) {
            largest = arr[i];
        }
        if (arr[i] < smallest) {
            smallest = arr[i];
        }
    }
    cout << "Largest: " << largest << endl;
    cout << "Smallest: " << smallest << endl;
}

int main() {
    int arr[] = {2, 1, 3, 1, 2};
    int n = 5;
    FindLargetAndSmallest(arr, n);
    return 0;
}
```

Bubble Sort :


BUBBLE SORTING
First Pass
Second Pass
Third Pass
Swapping
No Swap

Bubble Sort :

```
Repeat n-1 times
    For i from 0 to n-2
        If numbers[i] and numbers[i+1] out of order
            Swap them
    If no swaps
        Quit
```

(Pseudocode)

Bubble Sort	$O(n^2)$	$\Omega(n)$
	Worst Case	**Best Case**

Bubble Sort Code : (Example#1)

```cpp
#include <iostream>
using namespace std;

void BubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);
            }
        }
    }
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

int main() {
    int arr[] = {2, 1, 3, 1, 2};
    int n = 5;
    BubbleSort(arr, n);
    return 0;
}
```

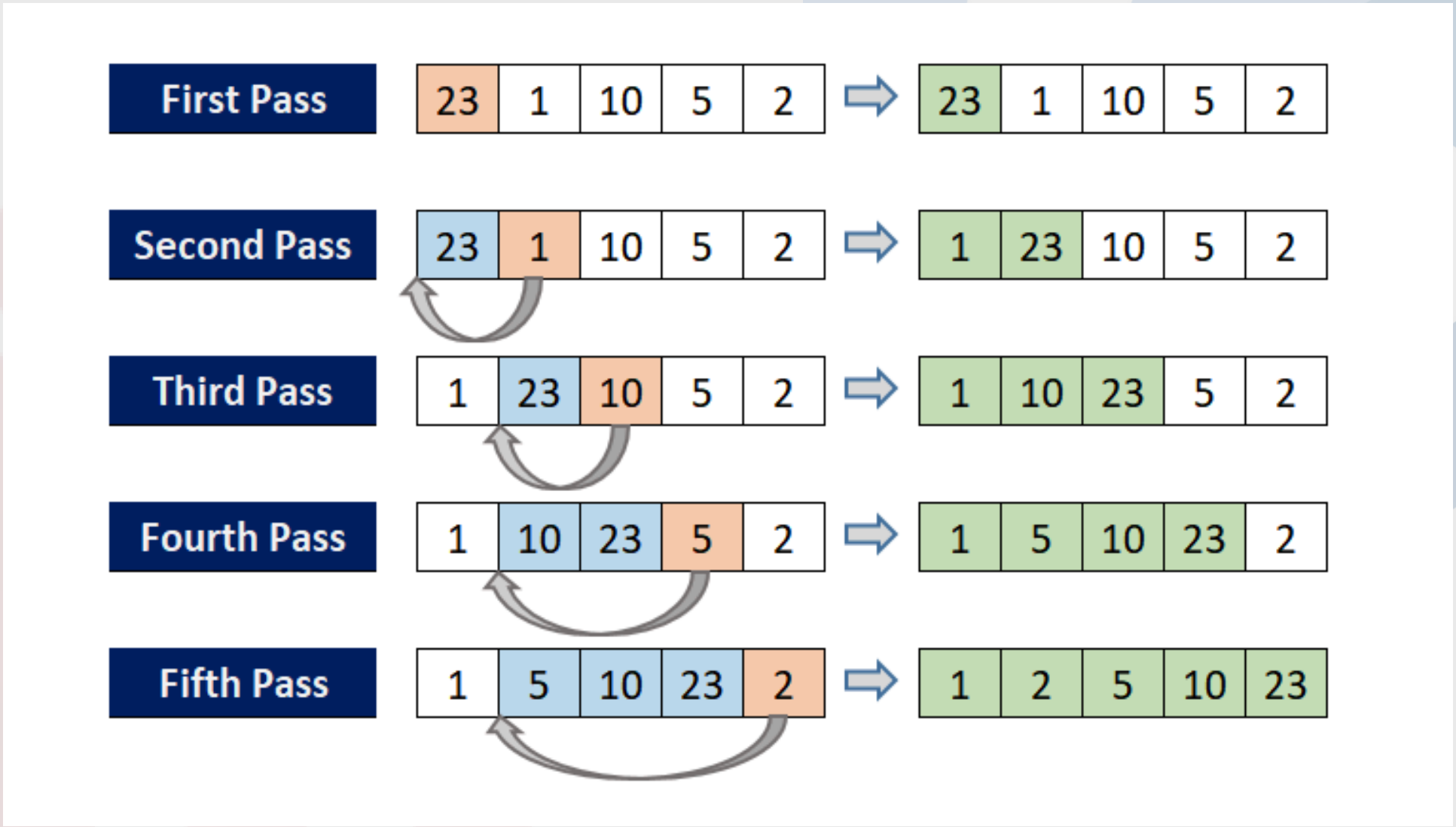
Bubble Sort : (Example#2)

```cpp
#include <iostream>
using namespace std;

bool isSorted(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        if (arr[i] < arr[i - 1]) {
            return false;
        }
    }
    return true;
}

int main() {
    int arr[] = {2, 1, 3, 1, 2};
    int n = 5;
    if (isSorted(arr, n)) {
        cout << "Yes" << endl;
    } else {
        cout << "No" << endl;
    }
    return 0;
}
```

Insertion Sort :



Insertion Sort :

```
1 For i in {2,…,n} do:
2   For j in {i,…,2} do:
3     If a(j)<a(j-1), swap a(j) and a(j-1);
4     Else break.
```

worst case: reverse-ordered elements in array.

$$\sum_{i=1}^{N-1} i = 1+2+3+\ldots+(N-1) = \frac{(N-1)N}{2}$$
$$= O(N^2)$$

best case: array is in sorted ascending order.

$$\sum_{i=1}^{N-1} 1 = N-1 = O(N)$$

average case: each element is about halfway in order.

$$\sum_{i=1}^{N-1} \frac{i}{2} = \frac{1}{2}(1+2+3\ldots+(N-1)) = \frac{(N-1)N}{4}$$
$$= O(N^2)$$

Insertion Sort : (Example#1)

```cpp
#include <iostream>
using namespace std;

void insertionSort(int arr[], int n)
{
    for (int i = 1; i < n; ++i) {
        int key = arr[i];
        int j = i - 1;

        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}

int main()
{
    int arr[] = { 12, 11, 13, 5, 6 };
    int n = 5;

    insertionSort(arr, n);

    return 0;
}
```

Insertion Sort : (Example#1)

Count Number of Swaps needed to sort an array using insertion sort

```cpp
#include <iostream>
using namespace std;

int insertionSortSwaps(int arr[], int n) {
    int swaps = 0;

    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;

        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j -= 1;
            swaps += 1;
        }

        arr[j + 1] = key;
    }
    return swaps;
}

int main() {
    int arr[] = {2, 1, 3, 1, 2};
    int n = 5;
    int swaps = insertionSortSwaps(arr, n);
    cout << swaps << endl;

    return 0;
}
```

Insertion Sort :

Algorithm	Time Complexity (Best)	Time Complexity (Average)	Time Complexity (Worst)	Space Complexity
Bubble Sort	O(n)	O(n^2)	O(n^2)	O(1)
Insertion Sort	O(n)	O(n^2)	O(n^2)	O(1)
Selection Sort	O(n^2)	O(n^2)	O(n^2)	O(1)

Insertion Sort :

Given an array, find and print all duplicate elements.

Input :
arr[] = {1, 2, 3, 2, 4, 3, 5, 1}
Output:
1 2 3

Support Team